DUOC UC

BDY0101 · Base de Datos No Relacionales

Evaluación Parcial N°2

Implementación de una base de datos NoSQL en MongoDB Atlas y manipulación de datos

Caso ContentFlow · Forma A · Informe técnico

Dato	Detalle
Integrante	Rodrigo Guajardo (equipo de una persona)
Docente	Matias Humberto Quinchen Ortiz
Sección	003V-OLS
Sede	Duoc UC, Sede Melipilla
Repositorio	https://github.com/YamikugoKurogane/contentflow-atlas
Fecha	12 de noviembre de 2026

1. Objetivos del proyecto

ContentFlow es una plataforma digital donde los usuarios publican contenido, reaccionan, comentan y participan en campañas. La gerencia necesita identificar a los usuarios más activos, los contenidos más populares y el impacto de las campañas, y que sus aplicaciones puedan consultar y registrar interacciones directamente en una base de datos NoSQL en la nube.

Objetivo general. Implementar en MongoDB Atlas la base de datos de ContentFlow a partir de los datos entregados, desarrollar consultas y operaciones CRUD que respondan a las necesidades de la gerencia e integrarla con una API REST que lea y escriba datos en tiempo real, con control de errores y trazabilidad.

Objetivos específicos.

- Configurar un cluster M0 en Atlas con usuario, acceso de red, niveles de consistencia y verificación de conectividad (IL2.1).
- Cargar los cinco archivos JSON del anexo, convertir sus fechas a tipo `Date` y detectar y tratar las inconsistencias de los datos (IL2.1, IL2.3).
- Desarrollar consultas de lectura y pipelines de agregación que respondan a requerimientos de negocio, y evaluar su eficiencia con `explain` (IL2.2).
- Implementar operaciones CRUD que simulen casos de negocio de ContentFlow (IL2.3).
- Integrar la base con una API REST propia, con validaciones, manejo de errores y registros de actividad (IL2.4).

2. Configuración de la base de datos en MongoDB Atlas

2.1 Cluster M0

Parámetro	Valor
Proyecto Atlas	Project 0
Nombre del cluster	`ClusterPrimario`
Tier	M0 (gratuito, requerido por el encargo)
Tipo	Replica Set de 3 nodos
Versión de MongoDB	8.0.37
Proveedor cloud y región	AWS, São Paulo (`sa-east-1`)
Host	`clusterprimario.fc8qtk7.mongodb.net`

Se eligió el tier M0 porque el encargo lo exige y porque el volumen del caso (2.500 documentos) cabe con holgura en sus 512 MB. Se utiliza la región São Paulo por su cercanía geográfica con Chile, lo que reduce la latencia de las conexiones. El replica set de 3 nodos es el que Atlas entrega por defecto y respalda los niveles de consistencia `majority` descritos en la sección 2.6.

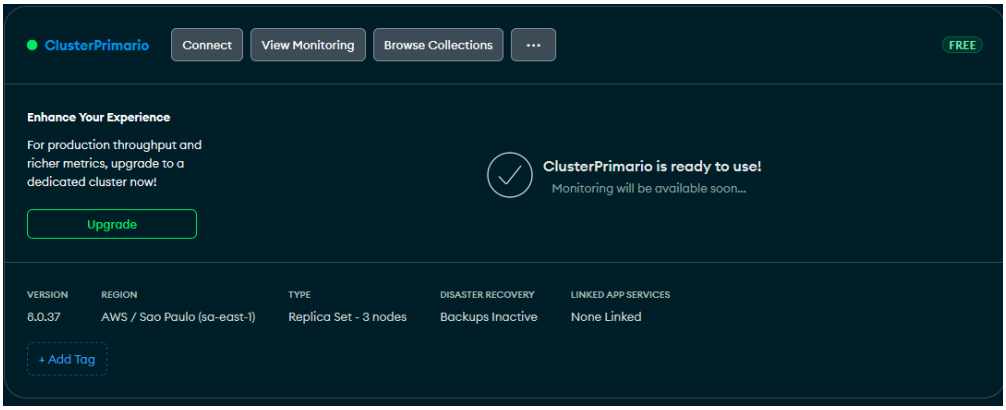


Figura 1. Cluster ClusterPrimario en Atlas: versión 8.0.37, región AWS São Paulo (sa-east-1) y Replica Set de 3 nodos.

2.2 Base de datos y colecciones

La base de datos del proyecto se llama `ClusterPrimario` y contiene las cinco colecciones del caso más una colección técnica de auditoría. Los `_id` se mantuvieron como texto (por ejemplo `user_1`), tal como indica el anexo. El cluster también incluye la base `sample_mflix`, que corresponde al conjunto de datos de ejemplo de Atlas y no forma parte del proyecto.

Colección	Documentos	Storage (Compass)	Descripción
usuarios	500	69,63 kB	Nombre, correo, país, género, fecha de registro y dispositivo
publicaciones	500	167,94 kB	Contenido, categoría, hashtags, tipo y fecha de publicación
reacciones	499	77,82 kB	Tipo de reacción por usuario y publicación (500 menos 1 duplicado eliminado)
comentarios	500	114,69 kB	Comentario, fecha y número de likes
campanas	500	77,82 kB	Campañas con fechas, objetivo, publicaciones asociadas y métricas
auditoria_correcciones	1	20,48 kB	Respaldo del documento eliminado en la corrección de datos (colección técnica)

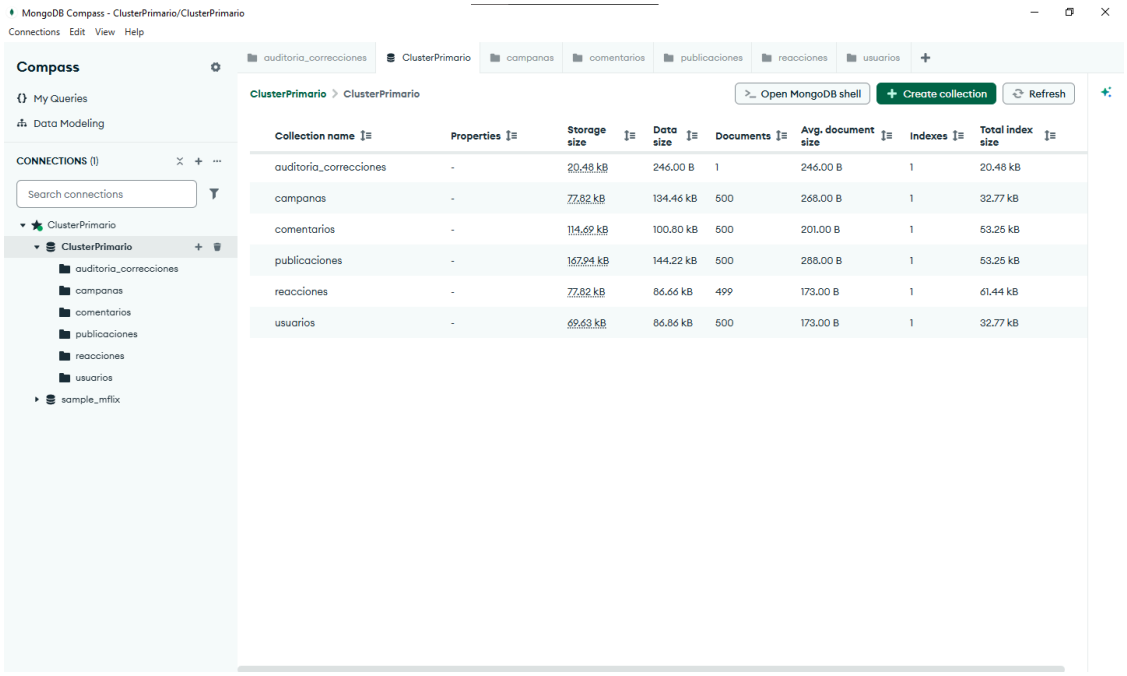


Figura 2. Colecciones de la base ClusterPrimario en MongoDB Compass, con conteo de documentos.

2.3 Usuarios de base de datos y acceso de red

Usuario	Rol en Atlas	Uso
REGI	`readWrite` sobre la base `ClusterPrimario`	Usuario de conexión de los scripts, Compass y la API
rorguajardo_db_user	`atlasAdmin@admin`	Administrador generado automáticamente al crear el cluster; no se usa en el código

Ajuste de permisos. Al inicio el usuario REGI tenía el rol `readWriteAnyDatabase@admin`, que da acceso de lectura y escritura a cualquier base del cluster. Como el encargo pide el rol `readWrite` sobre la base del proyecto, se restringió siguiendo el principio de mínimo privilegio. Después del cambio se ejecutaron los scripts 06 (creación de 9 índices en las 5 colecciones) y 07 (13 consultas), ambos sin errores de autorización, lo que valida que el nuevo rol es suficiente.

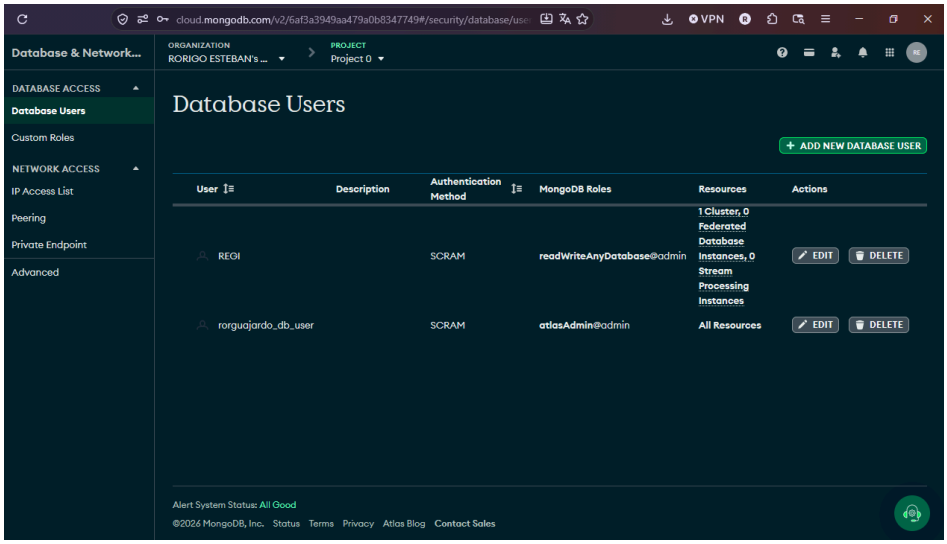


Figura 3. Database Users al inicio: REGI con el rol readWriteAnyDatabase.

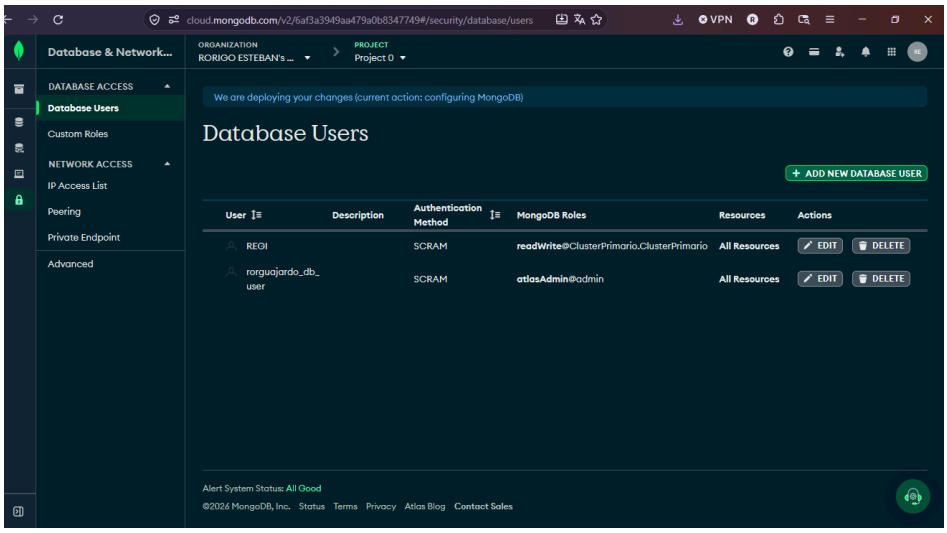


Figura 4. Database Users después del ajuste: REGI con el rol readWrite sobre ClusterPrimario.

Network Access. La lista de acceso contiene solo la IP pública del equipo de desarrollo (`45.238.21.170/32`), agregada durante la configuración inicial y con estado Active. No se abrió el acceso a `0.0.0.0/0`, de modo que solo se puede conectar desde esa dirección. Si la red cambia, hay que agregar la nueva IP.

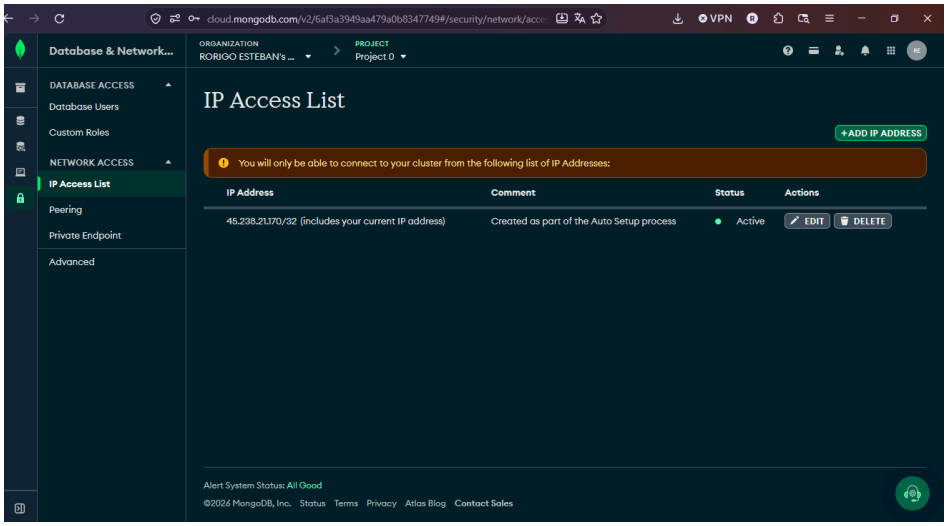


Figura 5. IP Access List en Atlas.

2.4 Índices

Los índices se crearon con el script `06_crear_indices.py` (evidencia en `docs/evidencias/06_indices.json`), después de medir las consultas sin índice (sección 4.3). Como el sharding no está disponible en M0, los índices son el mecanismo de optimización. Cada colección conserva además el índice `_id_` que MongoDB crea por defecto.

Colección	Índice	Claves	Consulta que lo justifica
usuarios	`ux_correo` (único)	`correo`	Búsqueda por correo y prevención de correos duplicados
usuarios	`ix_pais`	`pais`	Perfil de audiencia por país
publicaciones	`ix_pub_usuario`	`usuario_id`	Publicaciones de un usuario y `$lookup` (A1)
publicaciones	`ix_pub_categoria_fecha`	`categoria`, `fecha_publicacion` (desc)	Contenido por categoría y recencia (F1, T2)
reacciones	`ix_react_pub`	`publicacion_id`	Popularidad por publicación y `$lookup` (A2, A3, A7, T1)
reacciones	`ux_react_usuario_pub` (único)	`usuario_id`, `publicacion_id`	Impide reacciones duplicadas
comentarios	`ix_com_pub`	`publicacion_id`	Comentarios de una publicación y `$lookup` (A2, A7)
comentarios	`ix_com_usuario`	`usuario_id`	Usuarios más activos (A1, T3)
campanas	`ix_camp_pubs` (multikey)	`publicaciones_asociadas`	Campañas a las que pertenece una publicación

Los dos índices únicos se crearon sin error. Eso confirma que no hay correos repetidos y que la reacción duplicada detectada se eliminó correctamente antes (sección 5.3), ya que MongoDB habría rechazado el índice si aún existieran duplicados.

2.5 Límites del tier M0 y su impacto

Límite	Valor en M0	Impacto en el proyecto
Almacenamiento	512 MB (incluye índices)	El proyecto ocupa cerca de 0,5 MB (suma del storage de las colecciones en Compass), alrededor del 0,1 % del límite
Operaciones	Máximo 100 lecturas y escrituras por segundo	Suficiente para pruebas y demostración de la API; una carga masiva se vería limitada
Conexiones	Máximo 500 simultáneas según la documentación vigente de Atlas (documentación antigua indicaba 100)	La API usa un pool de conexiones acotado, por lo que no se acerca al límite
Sharding	No disponible	La optimización se hace con índices
Agregaciones	Sin `allowDiskUse`; sin JavaScript del lado del servidor (`$where`, map-reduce)	Los pipelines se resuelven en memoria; con 500 documentos por colección no hay riesgo
Inactividad	Atlas puede pausar el cluster tras 30 días sin actividad	Hay que reanudar el cluster desde Atlas si se pausa antes de la entrega

Fuente: documentación de MongoDB Atlas sobre clusters gratuitos (Free Cluster Limits). Los valores deben contrastarse con la versión vigente al momento de la entrega.

2.6 Nivel de consistencia: read concern y write concern

Todas las conexiones se crean con **read concern majority** y **write concern w: "majority"**, con `retryWrites` activado. Esta configuración está definida en el módulo `scripts/common.py`:

```
client = MongoClient(
    uri,
    serverSelectionTimeoutMS=8000,
    w="majority",
    readConcernLevel="majority",
    retryWrites=True,
)
```

Con `majority`, una escritura se confirma solo cuando la reconoce la mayoría de los nodos del replica set, y una lectura devuelve únicamente datos confirmados por la mayoría. Esto evita leer datos que luego podrían revertirse. Como costo, la latencia es algo mayor, aceptable para el volumen de este proyecto. La salida del script 01 confirma el read concern (`{'level': 'majority'}`).

2.7 Verificación de conectividad

La conectividad se verificó con cuatro herramientas distintas, todas funcionando de forma estable contra el mismo cluster:

Herramienta	Cómo se verificó	Resultado
Driver pymongo (Python)	Script `01_verificar_conexion.py`: `ping`, versión, colecciones y almacenamiento	`ping: ok`, versión 8.0.37
MongoDB Compass	Conexión con el string de Atlas y exploración de las 6 colecciones	Conexión exitosa, conteos correctos
Data Explorer de Atlas	Consultas con filtros sobre las colecciones desde el navegador (sección 6.4)	Documentos visibles y consultables
API REST (Flask) y Postman	Endpoint `GET /health` y colección de 16 solicitudes	Respuesta 200 con `"estado": "ok"`

Driver pymongo. Salida del script 01 (ejecutado antes de la importación, por eso la lista de colecciones está vacía):

```
00:35:41,598 [INFO] ping: ok
00:35:41,598 [INFO] version_servidor: 8.0.37
00:35:41,598 [INFO] base_de_datos: ClusterPrimario
00:35:41,599 [INFO] read_concern: {'level': 'majority'}
00:35:41,599 [INFO] colecciones: []
00:35:41,599 [INFO] almacenamiento_MB: 0.0
00:35:41,600 [INFO] limite_M0_MB: 512
```

MongoDB Compass. Conexión creada con el URI de Atlas (la contraseña queda oculta en la interfaz):

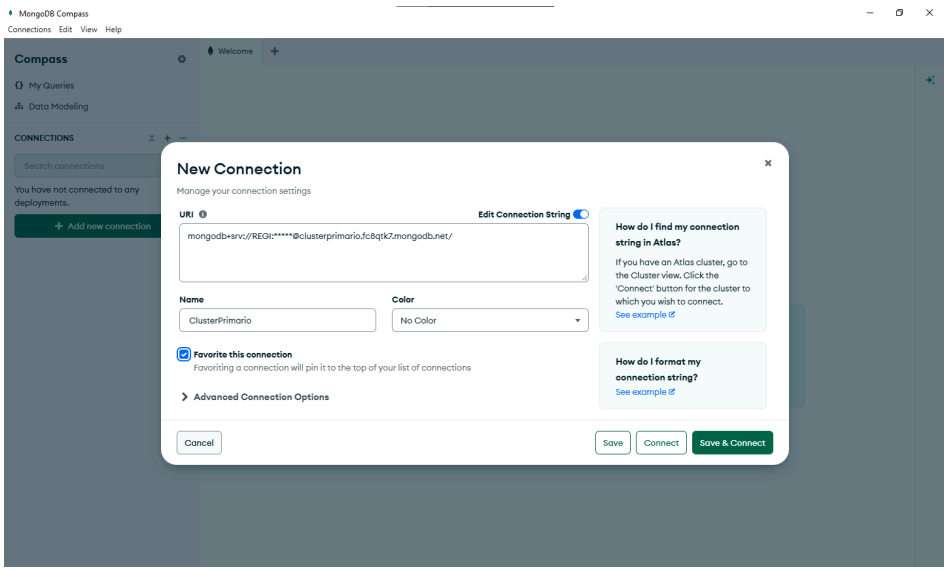


Figura 6. Conexión nueva en Compass hacia ClusterPrimario (contraseña oculta).

API REST. El endpoint `/health` ejecuta un `ping` a Atlas y devuelve el nombre de la base y la latencia. La primera llamada tarda cerca de un segundo (1.143 ms en el registro de la API y 1.044 ms en la captura), porque incluye abrir la conexión segura con el cluster. Las llamadas siguientes bajan a 55 a 61 ms, ya que reutilizan el pool de conexiones.

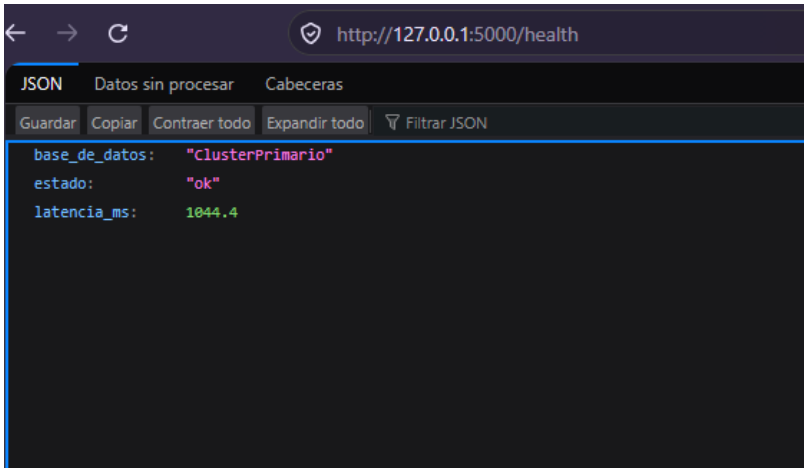


Figura 7. Respuesta de GET /health en el navegador: estado ok, base ClusterPrimario y latencia de la primera conexión.

2.8 Problemas encontrados y soluciones

Problema	Causa	Solución
Primer intento de conexión falló con `OperationFailure`	Error de autenticación: la contraseña se escribió mal al configurar la conexión con Atlas	Se corrigió la contraseña en el archivo `.env`. El siguiente intento respondió `ping: ok`
Advertencia: `pymongo 4.19.0 does not provide the extra 'srv'`	La versión actual de pymongo ya incluye el soporte SRV y el extra `[srv]` quedó obsoleto	No afecta el funcionamiento. En `requirements.txt` se dejó pymongo sin el extra
El archivo `campañas.json` traía la ñ en formato Unicode descompuesto	El nombre del archivo no coincidía con el esperado por los scripts	Se renombró a `campanas.json`, como indica el anexo
Fechas como texto	El anexo las entrega como strings ISO	Conversión a `Date` con `$toDate` (sección 3.2)
Rol del usuario demasiado amplio (`readWriteAnyDatabase`)	Era el rol que se había asignado al crear el usuario	Se cambió a `readWrite` sobre `ClusterPrimario` y se revalidó con los scripts 06 y 07 (sección 2.3)
Base de datos con el mismo nombre del cluster	Se usó `ClusterPrimario` como nombre de base	Se documenta en el informe; no afecta el funcionamiento

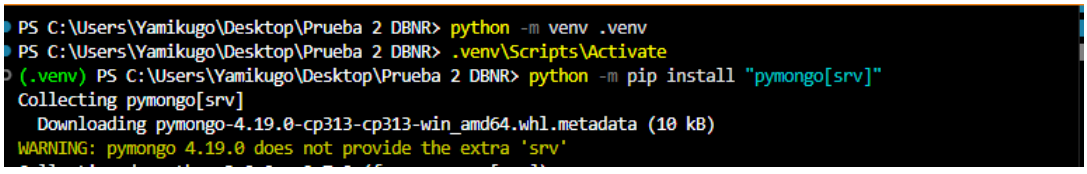


Figura 8. Creación del entorno virtual e instalación de pymongo, con la advertencia sobre el extra srv.

3. Carga e inserción de datos de prueba

3.1 Importación de los archivos JSON

Los cinco archivos del anexo son arreglos JSON. Se importaron con el script `02_importar_datos.py`, que equivale a `mongoimport --jsonArray`: lee cada archivo, inserta los documentos con `insert_many` y compara la cantidad cargada con la del archivo. Los `_id` se conservaron como texto.

```
00:36:01,533 [INFO] usuarios       archivo=500  atlas=500  OK
00:36:05,536 [INFO] reacciones     archivo=500  atlas=500  OK
00:36:07,533 [INFO] comentarios    archivo=500  atlas=500  OK
00:36:09,539 [INFO] campanas       archivo=500  atlas=500  OK
```

La línea de `publicaciones` no figura en el texto copiado de la consola, pero Compass confirma que la colección tiene 500 documentos. [Si conservas el log completo, reemplaza este bloque por la salida completa de logs/02_importar_datos.log.]

Colección	Documentos en el archivo	Documentos en Atlas	Resultado
usuarios	500	500	Coincide
publicaciones	500	500	Coincide (verificado en Compass)
reacciones	500	500 (499 tras la corrección)	Coincide en la carga
comentarios	500	500	Coincide
campanas	500	500	Coincide

3.2 Conversión de campos de fecha a tipo Date

Las fechas llegaron como texto en formato ISO. Para poder usar operadores como `$hour`, `$dayOfWeek` o `$month`, y comparar fechas entre colecciones, se convirtieron a `Date` con el script `03_convertir_fechas.py`. Usa una actualización con pipeline de agregación y solo toca los documentos cuyo campo aún es texto, por lo que se puede repetir sin riesgo.

```
col.update_many(
    {campo: {"$type": "string"}},
    [{"$set": {campo: {"$toDate": f"${campo}"}}}],
)
```

Colección.campo	Tipo antes	Modificados	Tipo después
usuarios.fecha_registro	string: 500	500	date: 500
publicaciones.fecha_publicacion	string: 500	500	date: 500
reacciones.fecha	string: 500	500	date: 500
comentarios.fecha	string: 500	500	date: 500
campanas.fecha_inicio	string: 500	500	date: 500
campanas.fecha_fin	string: 500	500	date: 500

3.3 Verificación de documentos en Compass

Las siguientes capturas muestran cada colección en Compass. En ellas se ve que las fechas aparecen como `ISODate(...)` (tipo `Date`) y que los `_id` conservan su formato de texto. Algunos documentos incluyen el campo `inconsistencia_temporal: true`, agregado en la corrección descrita en la sección 5.

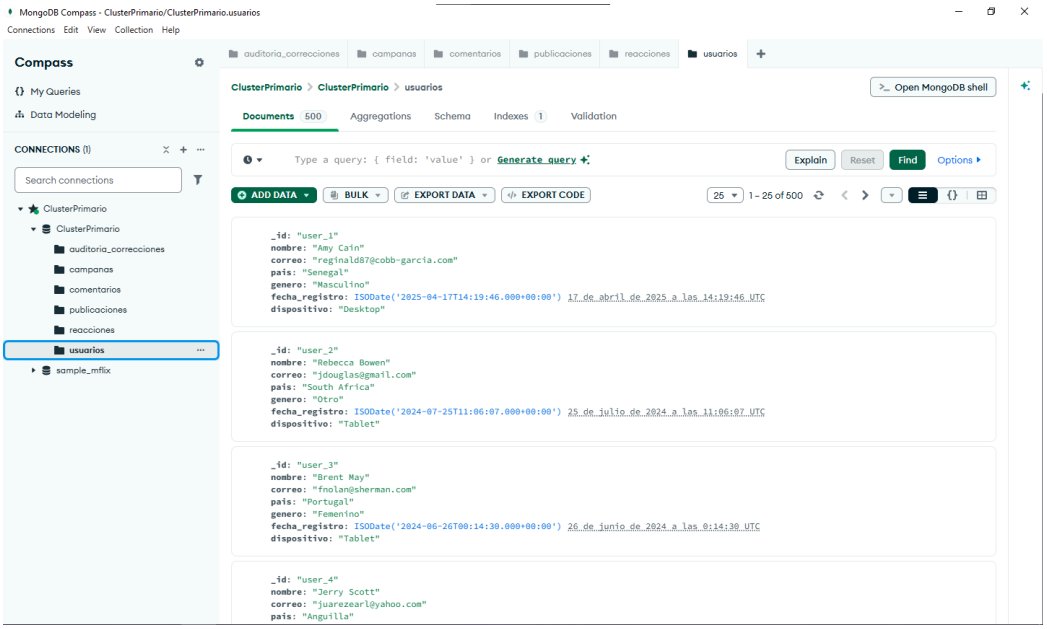


Figura 9. Colección usuarios (500 documentos).

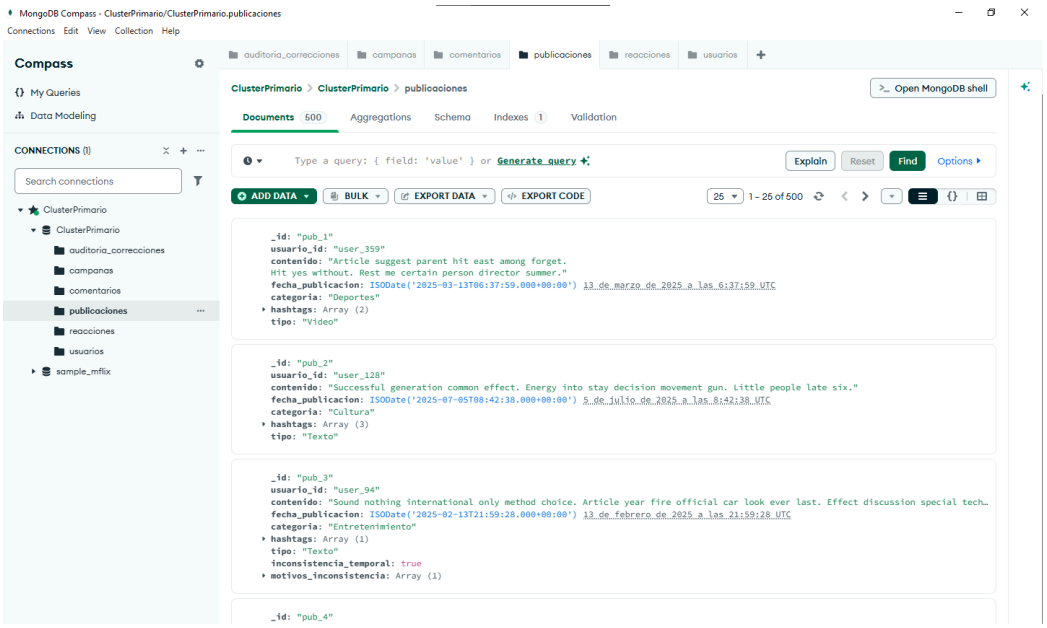


Figura 10. Colección publicaciones (500 documentos). pub_3 aparece marcada con inconsistencia_temporal.

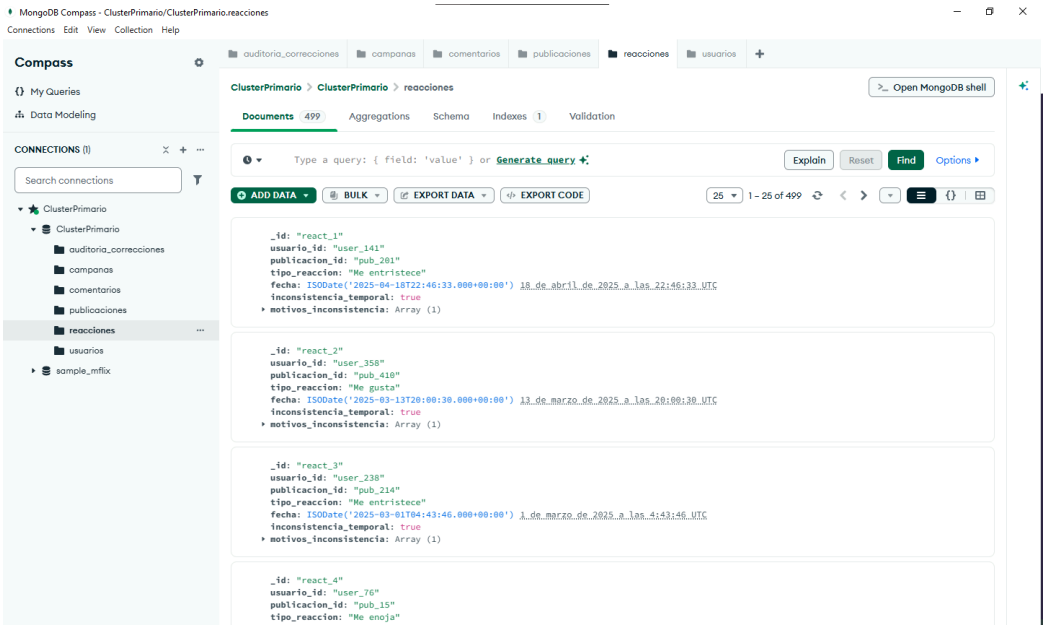


Figura 11. Colección reacciones (499 documentos tras eliminar el duplicado).

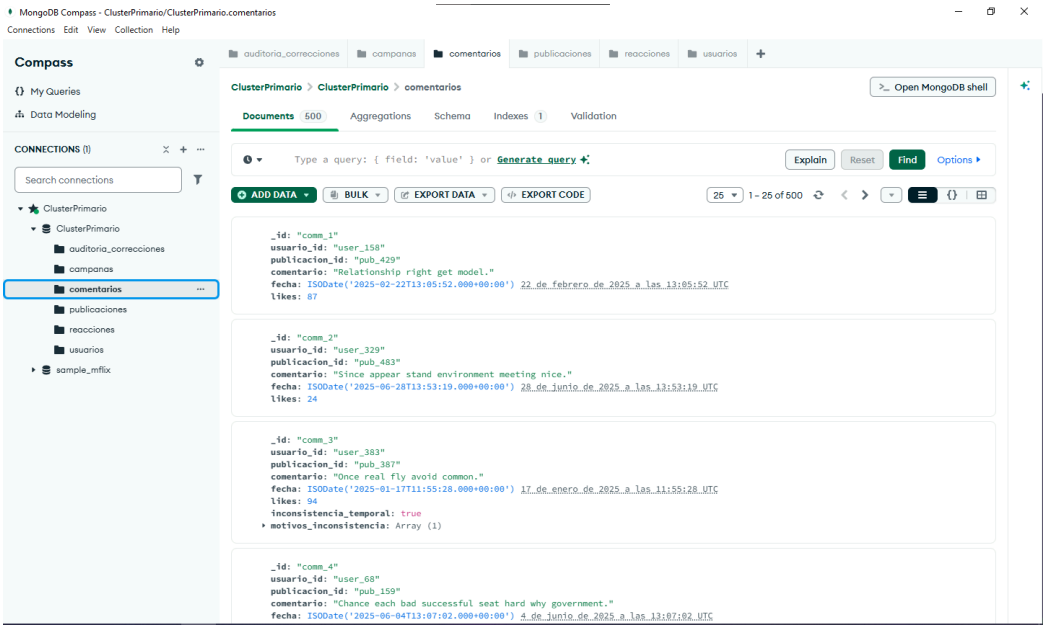


Figura 12. Colección comentarios (500 documentos). comm_3 aparece marcado con inconsistencia_temporal.

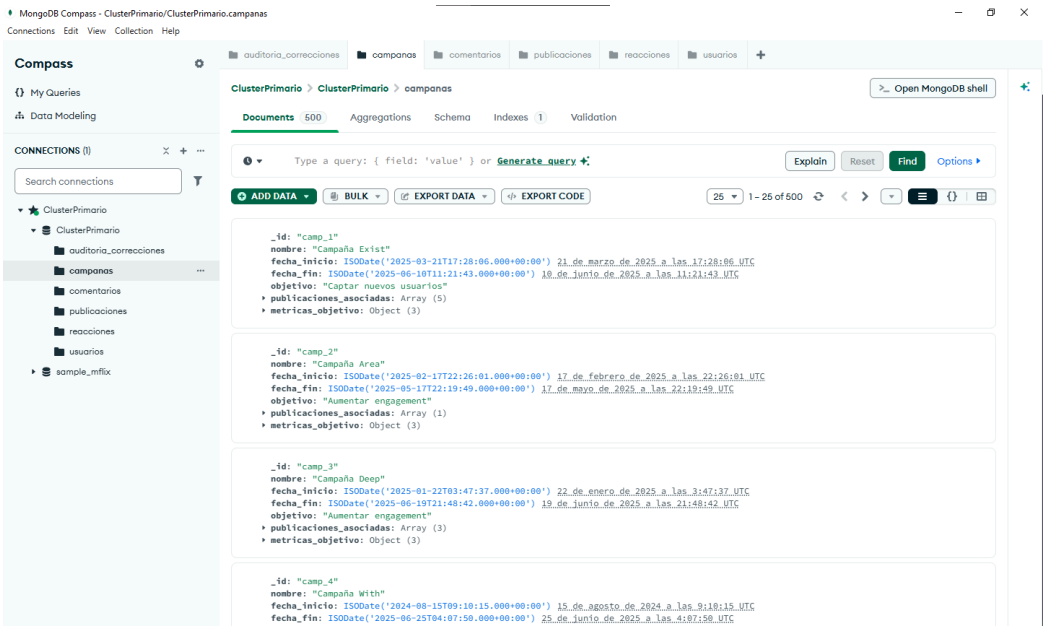


Figura 13. Colección campanas (500 documentos), con fechas de inicio y fin como Date.

Los ejemplos de inserción y verificación de documentos de prueba (usuario, publicación, reacciones y comentario) se ejecutan y documentan en la sección 5.4, con el estado antes y después de cada operación.

4. Desarrollo de consultas sobre la base de datos

Las consultas están en el script `07_consultas.py` y se ejecutaron contra Atlas con el usuario REGI. Las 13 consultas (5 de lectura y 8 de agregación) devolvieron resultados; la evidencia completa está en `docs/evidencias/07_consultas.json`. Todas comparan fechas como tipo `Date`, convertidas en la sección 3.2. Las consultas de lectura se limitaron a 5 documentos.

4.1 Consultas de lectura: filtros, proyecciones y ordenamientos

Id	Requerimiento de negocio	Consulta	Resultado
F1	Diseño de contenido: videos de Deportes más recientes	Filtro `categoria` = Deportes y `tipo` = Video; proyección de contenido, hashtags y fecha; orden por fecha descendente	La más reciente es `pub_338` (6 de julio de 2025)
F2	Perfil de audiencia: usuarios de móvil o tablet registrados desde 2025	`$and` de `dispositivo` con `$in` (Móvil, Tablet) y `fecha_registro` con `$gte` 2025-01-01; orden ascendente por fecha	El primero es Rick Jones, Tablet (6 de enero de 2025)
F3	Contenido popular: comentarios con 80 o más likes	`likes` con `$gte` 80; orden por likes descendente	Los primeros tienen 100 likes: `comm_321`, `comm_345`, `comm_435` y `comm_59`
F4	Búsqueda compuesta: publicaciones sobre fútbol o series que no sean texto	`$or` de hashtags #futbol o #series, `tipo` con `$ne` Texto y `inconsistencia_temporal` con `$ne` true	Devuelve `pub_1`, `pub_10`, `pub_100`, `pub_101`, entre otras

Id	Requerimiento de negocio	Consulta	Resultado
F5	Éxito de campañas: campañas vigentes el 1 de mayo de 2025	`fecha_inicio` con `$lte` y `fecha_fin` con `$gte` 2025-05-01	La primera es `camp_255`, Campaña Beat (16 de julio de 2024 al 5 de mayo de 2025)

4.2 Pipelines de agregación

Id	Requerimiento de negocio	Etapas	Resultado destacado
A1	Usuarios más activos (publicaciones + reacciones + comentarios)	`$lookup` ×3, `$project`, `$addFields`, `$sort`, `$limit`	Lidera `user_311` (Michael Bass, Botswana) con 10 interacciones
A2	Contenidos más populares (reacciones + comentarios)	`$lookup` ×2, `$project`, `$addFields`, `$sort`, `$limit`	`pub_237` (Deportes, Imagen) con popularidad 8: 3 reacciones y 5 comentarios
A3	Mejor hora de publicación	`$lookup`, `$project` con `$hour`, `$group`, `$sort`	Las 14:00 UTC, con 1,45 reacciones promedio en 22 publicaciones
A4	Mejor día de la semana	`$lookup`, `$project` con `$dayOfWeek`, `$group`, `$sort`	El miércoles (día 4), con 1,28 reacciones promedio en 68 publicaciones
A5	Perfil de audiencia: reacciones según dispositivo	`$lookup`, `$unwind`, `$group`, `$sort`	En móvil, Me entristece y Me enoja empatan con 55 reacciones cada una
A6	Hashtags más usados	`$unwind`, `$group`, `$sort`, `$limit`	#cultura lidera con 221 usos
A7	Éxito de campañas frente a su meta	`$lookup` ×2, `$project`, `$addFields` con `$round`, `$sort`, `$limit`	La mejor, `camp_239`, logra 5,6 % de su meta de reacciones
A8	Éxito de campañas por objetivo	`$lookup`, `$project`, `$group`, `$project`, `$sort`	Cada objetivo promedia 2,8 a 2,9 reacciones frente a una meta cercana a 555

A continuación se muestran dos de los pipelines (en sintaxis de mongosh; el script los ejecuta con pymongo):

A1. Usuarios más activos

```
db.usuarios.aggregate([
  {$lookup: {from: "publicaciones", localField: "_id", foreignField: "usuario_id", as: "pubs"}},
  {$lookup: {from: "reacciones", localField: "_id", foreignField: "usuario_id", as: "reacs"}},
  {$lookup: {from: "comentarios", localField: "_id", foreignField: "usuario_id", as: "coms"}},
  {$project: {nombre: 1, pais: 1, publicaciones: {$size: "$pubs"},
              reacciones: {$size: "$reacs"}, comentarios: {$size: "$coms"}}},
  {$addFields: {actividad_total: {$add: ["$publicaciones", "$reacciones", "$comentarios"]}}},
  {$sort: {actividad_total: -1, _id: 1}},
  {$limit: 10}
])
```

A7. Éxito de campañas

```
db.campanas.aggregate([
  {$lookup: {from: "reacciones", localField: "publicaciones_asociadas",
             foreignField: "publicacion_id", as: "r"}},
```

```
  {$lookup: {from: "comentarios", localField: "publicaciones_asociadas",
             foreignField: "publicacion_id", as: "c"}},
  {$project: {nombre: 1, objetivo: 1, meta_likes: "$metricas_objetivo.likes",
              meta_comentarios: "$metricas_objetivo.comentarios",
              reacciones: {$size: "$r"}, comentarios: {$size: "$c"}}},
  {$addFields: {cumplimiento_reacciones_pct: {$round: [
      {$multiply: [{$divide: ["$reacciones", "$meta_likes"]}, 100]}, 1]}}},
  {$sort: {cumplimiento_reacciones_pct: -1, _id: 1}},
  {$limit: 5}
])
```

4.3 Lectura de los resultados para el negocio

- **Usuarios más activos (A1).** El top 10 va de 10 a 7 interacciones por usuario. No hay usuarios que dominen la actividad: la participación es baja y pareja, por lo que una estrategia de retención debería apuntar a elevar la actividad de la base completa y no solo de unos pocos.
- **Contenido popular (A2).** De las 10 publicaciones más populares, 5 son videos, 3 imágenes y 2 textos. Deportes y Cultura aparecen tres veces cada una. El video es el formato que más repite entre los contenidos de mayor interacción.
- **Horarios (A3 y A4).** Las mejores horas (14, 13, 21 y 8 UTC) promedian entre 1,37 y 1,45 reacciones, y el mejor día es el miércoles con 1,28 frente a 0,84 del mínimo. Las diferencias son pequeñas, así que sirven como orientación y no como una regla firme. Atlas calcula la hora en UTC; las 14:00 UTC equivalen a las 11:00 en Chile durante el horario de verano.
- **Audiencia (A5).** En móvil, las reacciones negativas (Me entristece y Me enoja, 55 cada una) superan a las positivas (Me gusta 41 y Me encanta 40). En escritorio y tablet, Me entristece y Me gusta quedan casi empatadas. Conviene revisar qué tipo de contenido genera enojo y tristeza en móvil.
- **Hashtags (A6).** #cultura (221), #aprendizaje (214), #innovacion (201), #futbol (197) y #series (185). Sirven para priorizar temas en el diseño de contenido.
- **Campañas (A7 y A8).** Ninguna campaña se acerca a su meta. La mejor logra 5,6 % de su meta de reacciones, y por objetivo el promedio es de 2,8 a 2,9 reacciones frente a metas de unas 555, es decir, alrededor de 0,5 %. Las metas están cientos de veces por encima de lo observado, por lo que habría que revisar si fueron definidas con otra escala o si los datos cargados son solo una muestra. Para la comparación se usaron todas las reacciones como equivalente de likes, y la meta de compartidos no se pudo medir porque el anexo no incluye datos de compartidos.

4.4 Evaluación de eficiencia con explain

El script `08_explain_indices.py` mide tres consultas con `explain("executionStats")`: primero sin índice, y luego con el índice creado. Cada medición se repite 5 veces y se informa la mediana del tiempo. La evidencia está en `docs/evidencias/08_explain_indices.json`.

Prueba	Consulta	Fase	Plan	Tiempo (ms)	Docs examinados	Claves	Retornados
T1	reacciones: `publicacion_id` = pub_430	Sin índice	COLLSCAN	0	499	0	4
T1		Con índice	IXSCAN	0	4	4	4
T2	publicaciones: categoría Tecnología, orden por fecha desc	Sin índice	COLLSCAN + SORT en memoria	0	500	0	111
T2		Con índice	IXSCAN, sin SORT	0	111	111	111
T3	comentarios: `usuario_id` = user_92	Sin índice	COLLSCAN	0	500	0	4
T3		Con índice	IXSCAN	0	4	4	4

Prueba	Índice usado	Examinados / retornados antes	Examinados / retornados después	Reducción de documentos examinados
T1	`ix_react_pub`	499 / 4 = 124,8	4 / 4 = 1,0	99,2 %
T2	`ix_pub_categoria_fecha`	500 / 111 = 4,5	111 / 111 = 1,0	77,8 %
T3	`ix_com_usuario`	500 / 4 = 125,0	4 / 4 = 1,0	99,2 %

Análisis. El tiempo medido fue de 0 ms con y sin índice (las cinco repeticiones dieron 0 ms antes y 1, 0, 0, 0, 0 ms después; el 1 ms inicial corresponde a la primera ejecución con el plan nuevo). Con solo 500 documentos por colección, el motor resuelve cualquiera de las consultas por debajo de la resolución de 1 ms, así que el tiempo no permite distinguir las dos situaciones. La mejora se ve en el trabajo realizado: sin índice se revisa toda la colección (COLLSCAN) y con índice solo los documentos necesarios (IXSCAN). En T1 y T3 se pasa de revisar casi 500 documentos para devolver 4 a revisar exactamente 4. En T2, además, el índice compuesto elimina la etapa de ordenamiento en memoria, porque ya entrega los documentos ordenados por fecha.

Un recorrido completo crece con el tamaño de la colección, mientras que un recorrido por índice crece con el tamaño del resultado. Con los miles de interacciones que menciona el caso, la diferencia pasaría de ser invisible a ser medible en tiempo.

Mejoras propuestas.

- Mantener los índices sobre los campos que usan los `$lookup` (`usuario_id` y `publicacion_id`), porque los pipelines A1, A2, A3 y A7 los recorren completos.
- Para escalar, precalcular el conteo de reacciones y comentarios por publicación (por ejemplo, como campos con `$inc` al insertar), y así evitar `$lookup` masivos en las consultas de popularidad.

- Evaluar también los pipelines de agregación con `explain` cuando el volumen crezca; en esta entrega la medición se hizo sobre las consultas de lectura.
- Mantener el límite de resultados en la API (máximo 100) y proyectar solo los campos necesarios, para respetar el tope de 100 operaciones por segundo del tier M0.

5. Operaciones CRUD completas y calidad de datos

5.1 Detección de inconsistencias

Las inconsistencias se detectaron con consultas de agregación (script `04_validar_datos.py`) y no se eliminó ningún registro sin dejar evidencia. Se ejecutó el chequeo antes y después de la corrección.

Chequeo	Casos	Después	Tratamiento
Reacciones duplicadas (mismo usuario y publicación)	1	0	Corregido: se eliminó una, con respaldo
Reacciones con fecha anterior a su publicación	245	245	Marcadas con `inconsistencia_temporal`
Comentarios con fecha anterior a su publicación	251	251	Marcados con `inconsistencia_temporal`
Publicaciones anteriores al registro de su autor	48	48	Marcadas con `inconsistencia_temporal`
Reacciones anteriores al registro del usuario	53	53	Marcadas con `inconsistencia_temporal`
Nombres de campaña repetidos	91	91	Justificado: cada campaña se identifica por su `_id`
Autoreacciones y autocomentarios	1 y 1	1 y 1	Justificado: comportamiento válido en una red social
Publicaciones sin campaña	22	22	Justificado: no toda publicación participa en una campaña
Usuarios sin actividad	31	31	Justificado: usuarios registrados que aún no interactúan
Referencias inválidas (usuario, publicación o campaña inexistente)	0	0	Sin hallazgos
Campañas con fecha de fin anterior al inicio	0	0	Sin hallazgos

Las 245 reacciones y 251 comentarios con fecha anterior a su publicación, junto con los casos anteriores al registro del usuario, son demasiado numerosos para tratarse como errores aislados. Son consistentes con datos sintéticos generados sin restricción de orden cronológico.

5.2 Criterios de corrección

- **Duplicados:** se conserva un documento de cada par y el otro se respalda en `auditoria_correcciones` antes de eliminarlo.

- **Incoherencias temporales:** no se eliminan, porque el resto de cada documento sigue siendo útil para las consultas. Se marcan con `inconsistencia_temporal: true` y un arreglo `motivos_inconsistencia` con el chequeo que las detectó, de modo que se puedan filtrar.
- **Casos válidos o de contexto:** no se modifican y se justifican en este informe.

5.3 Evidencia del antes y el después

El script `05_corregir_datos.py` funciona primero en modo simulación y luego con `--aplicar`:

```
00:36:40,683 [INFO] Modo: SIMULACIÓN (dry-run)
00:36:41,563 [INFO] Reacciones duplicadas: 1 grupo(s) -> eliminar ['react_65']
00:36:41,710 [INFO] reacciones_antes_de_publicacion: 245 documento(s) a marcar
00:36:41,861 [INFO] comentarios_antes_de_publicacion: 251 documento(s) a marcar
00:36:41,947 [INFO] publicaciones_antes_de_registro: 48 documento(s) a marcar
00:36:42,037 [INFO] reacciones_antes_de_registro: 53 documento(s) a marcar
00:36:42,037 [INFO] Nada se modificó. Repite con --aplicar para ejecutar.

00:36:54,016 [INFO] Modo: APLICANDO
00:36:54,818 [INFO] Reacciones duplicadas: 1 grupo(s) -> eliminar ['react_65']
00:36:55,046 [INFO] Eliminadas: 1 (respaldo en auditoria_correcciones)
00:36:55,318 [INFO] reacciones_antes_de_publicacion: 245 documento(s) a marcar
00:36:55,407 [INFO]   marcados: 245
00:36:55,558 [INFO] comentarios_antes_de_publicacion: 251 documento(s) a marcar
00:36:55,648 [INFO]   marcados: 251
00:36:55,733 [INFO] publicaciones_antes_de_registro: 48 documento(s) a marcar
00:36:55,804 [INFO]   marcados: 48
00:36:55,890 [INFO] reacciones_antes_de_registro: 53 documento(s) a marcar
00:36:55,963 [INFO]   marcados: 53
```

Resultado verificable en Compass: la colección `reacciones` pasó de 500 a 499 documentos, y el documento eliminado quedó respaldado en `auditoria_correcciones`:

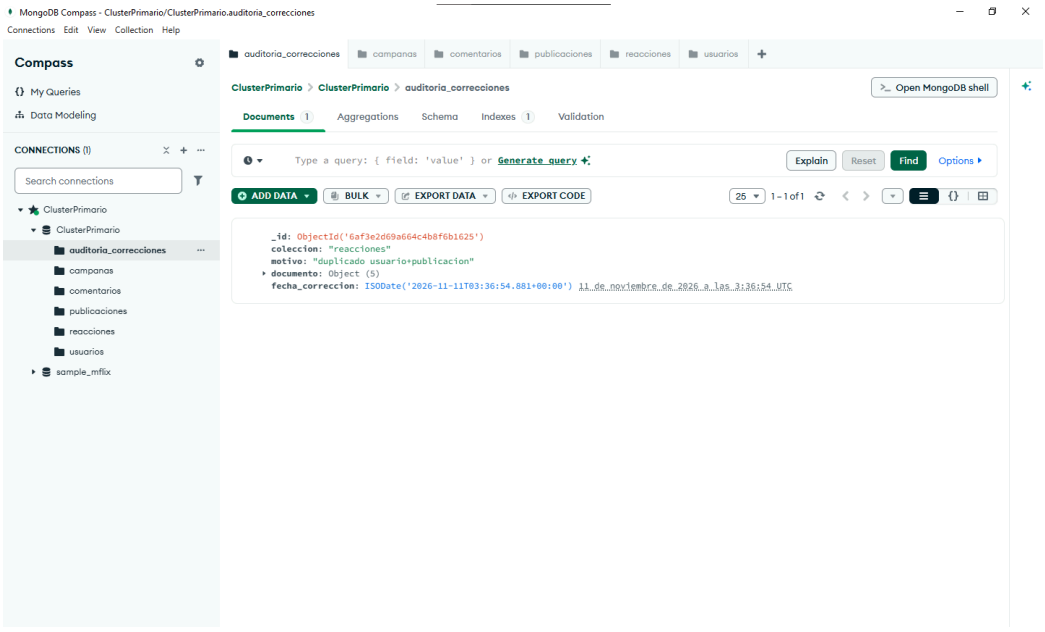


Figura 14. Documento de respaldo de react_65 en auditoria_correcciones.

Al repetir el chequeo después de aplicar la corrección, `reacciones_duplicadas` bajó de 1 a 0 y los demás conteos se mantuvieron, como se esperaba, porque los casos temporales se marcaron y no se eliminaron.

5.4 Script CRUD de un caso de negocio

El script `09_crud.py` simula el caso "lanzamiento de una publicación dentro de una campaña": un usuario nuevo se registra y publica, la publicación se asocia a la campaña `camp_1`, otros usuarios reaccionan y comentan, se actualizan las métricas y, al final, se eliminan los datos de prueba. Todos los documentos creados tienen `_id` con prefijo `demo_`, de modo que el script se puede repetir. Cada paso registra el estado antes y después (evidencia en `docs/evidencias/09_crud.json`).

Paso	Operación	Antes	Después
C1	`insert_one` en usuarios: registro de un usuario nuevo	No existía	`demo_user_1`, Usuario Demo, Chile, Móvil
C2	`insert_one` en publicaciones: el usuario publica	0 publicaciones del usuario	`demo_pub_1`, "Lanzamiento de la campaña de otoño", #innovacion, Texto
U1	`update_one` con `$push` en campanas: asociar la publicación a `camp_1`	5 publicaciones asociadas	6 (la última es `demo_pub_1`)
C3	`insert_many` en reacciones, validadas contra usuarios y publicaciones	0 reacciones de la publicación	2 reacciones
C4	`insert_one` en comentarios	0 comentarios	`demo_comm_1`, "Muy buena iniciativa", 0 likes
R1	`aggregate`: reacciones de la publicación agrupadas por tipo	-	Me encanta: 1; Me gusta: 1
R2	`aggregate` con `$lookup` y `$unwind`: publicación con los datos de su autor	-	`demo_pub_1`, autor Usuario Demo, Chile
U2	`update_one` con `$inc` y `$set` en comentarios	0 likes; "Muy buena iniciativa"	5 likes; "Muy buena iniciativa (editado)"
U3	`update_one` con `$push` y `$set` en publicaciones	Hashtags [#innovacion]; tipo Texto	Hashtags [#innovacion, #otono]; tipo Video
U4	`update_many` en reacciones: marcar como revisadas	2 sin marca	2 modificadas (`revisada: true`)
V1	Reacción a una publicación inexistente	Sin cambios	Rechazada: la publicación `pub_999999` no existe
V2	Reacción duplicada (mismo usuario y publicación)	Sin cambios	Rechazada: `user_1` ya reaccionó a `demo_pub_1`
V3	Reacción de un usuario inexistente	Sin cambios	Rechazada: el usuario `user_999999` no existe
D1	`delete_one` en comentarios	1 comentario	0 comentarios
D2	`delete_many` en reacciones de la publicación (evita referencias huérfanas)	2 reacciones	2 eliminadas

Paso	Operación	Antes	Después
D3	`$pull` en campanas y `delete_one` de la publicación y del usuario de prueba	Existían la publicación y el usuario	Publicación: 0; usuario: 0; presencia en la campaña: 0

Validaciones aplicadas. Antes de insertar una reacción, la función `validar_reaccion` comprueba que el usuario y la publicación existan y que no haya ya una reacción del mismo usuario sobre esa publicación. Los pasos V1, V2 y V3 muestran que las tres situaciones se rechazan sin modificar la base, que es el criterio aplicado para evitar referencias inválidas y duplicados.

```
def validar_reaccion(db, usuario_id, publicacion_id):
    if not db.usuarios.find_one({"_id": usuario_id}, {"_id": 1}):
        raise ErrorValidacion(f"referencia inválida: el usuario {usuario_id} no existe")
    if not db.publicaciones.find_one({"_id": publicacion_id}, {"_id": 1}):
        raise ErrorValidacion(f"referencia inválida: la publicación {publicacion_id} no existe")
    if db.reacciones.find_one({"usuario_id": usuario_id, "publicacion_id": publicacion_id}, {"_id": 1}):
        raise ErrorValidacion(f"duplicado: {usuario_id} ya reaccionó a {publicacion_id}")
```

Resultado final. El script compara los conteos antes y después de ejecutarse, y la base quedó intacta:

Colección	Conteo inicial	Conteo final
usuarios	500	500
publicaciones	500	500
reacciones	499	499
comentarios	500	500
campanas	500	500

6. Integración con aplicación externa (API REST)

6.1 Flujo de integración

La API REST es propia y está desarrollada en Python con Flask. Recibe solicitudes HTTP en formato JSON, valida los datos y opera sobre MongoDB Atlas con el driver oficial pymongo. Los clientes, como Postman o una aplicación de ContentFlow, nunca hablan directamente con la base de datos.

```
Cliente (Postman / aplicación)
        |  HTTP + JSON  (http://127.0.0.1:5000)
        v
API REST Flask  (api/app.py)
        |  validación de entradas -> pymongo (pool de conexiones, w=majority)
        v
MongoDB Atlas  (cluster ClusterPrimario, AWS São Paulo)
```

6.2 Librerías y manejo del string de conexión

Librería	Uso
Flask	Servidor HTTP, rutas, manejo de errores y registro de solicitudes

Librería	Uso
pymongo	Driver oficial de MongoDB para consultas, inserciones, actualizaciones y agregaciones
python-dotenv	Carga de las variables de entorno desde el archivo `.env`

El string de conexión se lee únicamente de la variable de entorno `MONGODB_URI`, definida en un archivo `.env` local que está excluido del repositorio con `.gitignore`. El repositorio incluye `.env.example` sin datos reales. El cliente se crea una sola vez y reutiliza su pool de conexiones, con `w="majority"`, `readConcernLevel="majority"` y `retryWrites` activado.

```
def get_db():
    global _client, _db
    if _db is None:
        uri = os.getenv("MONGODB_URI")
        if not uri:
            raise ApiError(503, "Falta la variable de entorno MONGODB_URI")
        _client = MongoClient(uri, serverSelectionTimeoutMS=5000, maxPoolSize=10,
                              w="majority", readConcernLevel="majority", retryWrites=True)
        _db = _client[os.getenv("DB_NAME", "contentflow")]
    return _db
```

6.3 Endpoints y funciones

Método y ruta	Colección	Función
GET `/health`	-	Prueba de conectividad con Atlas (`ping`)
GET `/publicaciones`	publicaciones	Lista con filtros por categoría, tipo y usuario, y límite
GET `/publicaciones/<id>`	publicaciones, reacciones, comentarios	Lee una publicación e incluye su conteo de reacciones y comentarios
POST `/publicaciones`	publicaciones, usuarios	Crea una publicación validando que el autor exista
PATCH `/publicaciones/<id>`	publicaciones	Edita contenido, categoría, tipo o hashtags con `$set`
DELETE `/publicaciones/<id>`	publicaciones, reacciones, comentarios, campanas	Elimina la publicación y, en cascada, sus reacciones y comentarios, y la quita de las campañas
GET y POST `/reacciones`	reacciones	Lista y crea reacciones (409 si ya existe, 422 si la referencia no existe)
DELETE `/reacciones/<id>`	reacciones	Elimina una reacción
GET y POST `/comentarios`	comentarios	Lista y crea comentarios
POST `/comentarios/<id>/like`	comentarios	Suma un like de forma atómica con `$inc`
DELETE `/comentarios/<id>`	comentarios	Elimina un comentario
GET `/campanas`	campanas	Lista campañas con sus métricas objetivo
GET `/estadisticas/publicaciones-populares`	publicaciones, reacciones, comentarios	Agregación de popularidad (mismo criterio que A2)

La API opera sobre tres colecciones con lectura y escritura (publicaciones, reacciones y comentarios), más lectura sobre campañas, con lo que cumple el mínimo de dos colecciones.

6.4 Evidencia de lectura y escritura en tiempo real

Se importó la colección de Postman `postman/ContentFlow_API.postman_collection.json`, que contiene 16 solicitudes con pruebas automáticas. Se ejecutó con el Collection Runner y las 16 pruebas pasaron.

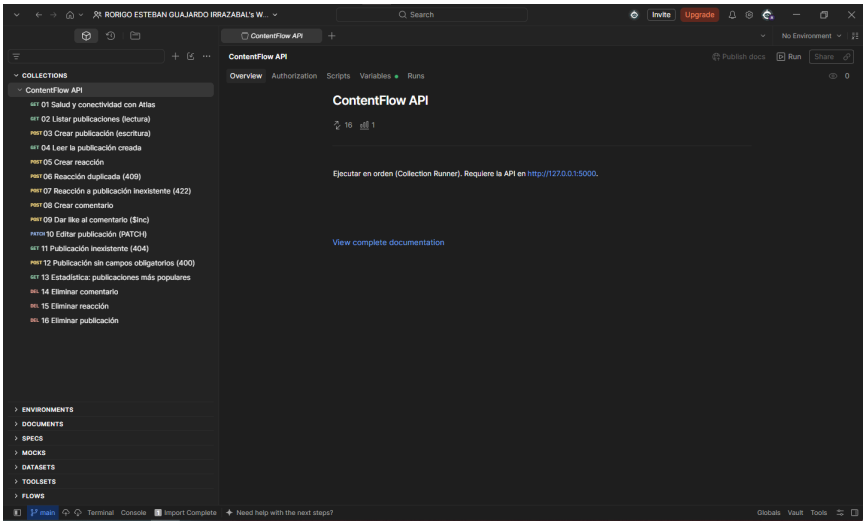


Figura 15. Colección ContentFlow API importada en Postman con sus 16 solicitudes.

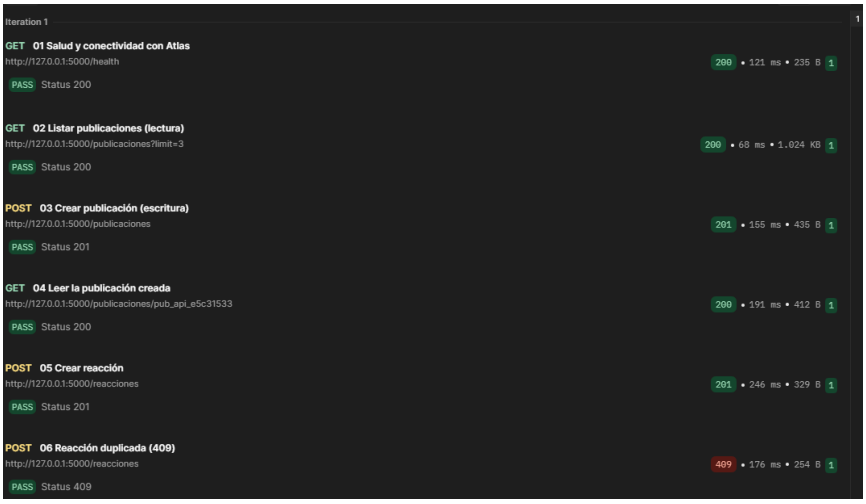


Figura 16. Collection Runner, solicitudes 01 a 06: salud, lectura, creación de publicación y reacción, y reacción duplicada (409).

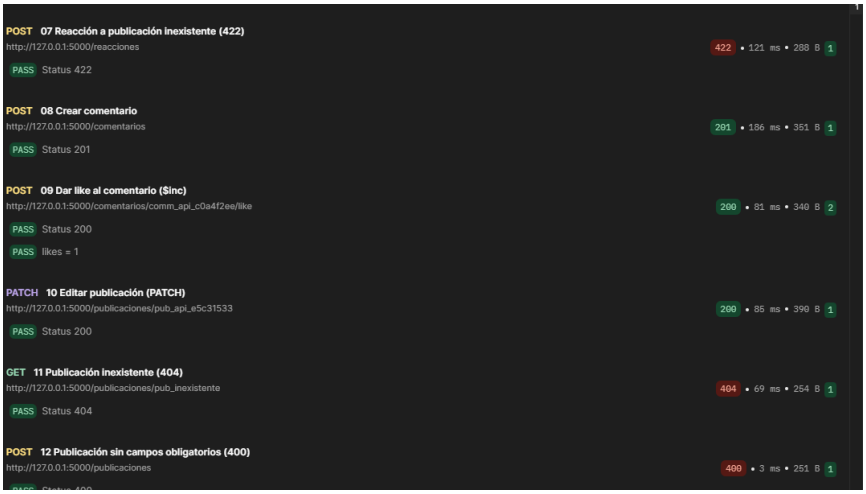


Figura 17. Collection Runner, solicitudes 07 a 12: referencia inválida (422), comentario, like con $inc, edición, publicación inexistente (404) y datos incompletos (400).

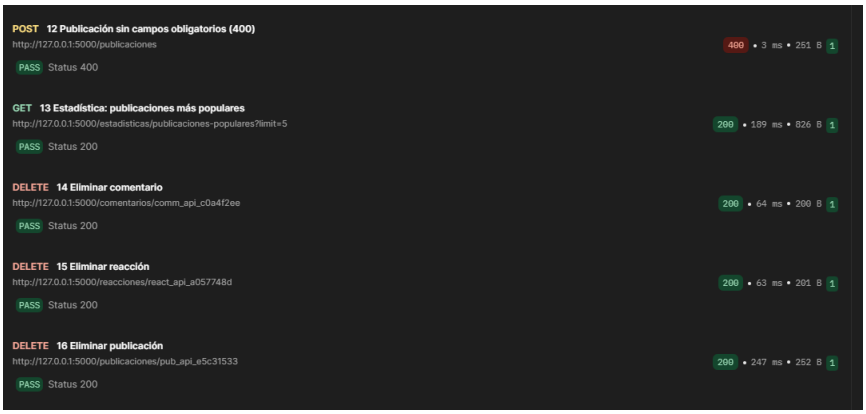


Figura 18. Collection Runner, solicitudes 12 a 16: estadística de popularidad y eliminación de comentario, reacción y publicación.

Reflejo en Atlas. Para demostrar que la API escribe en la base en tiempo real, se ejecutaron las solicitudes de la 01 a la 13 y se consultó Atlas con el Data Explorer, filtrando por los `_id` que genera la API (`pub_api_`, `react_api_` y `comm_api_`). Los documentos aparecen con los valores enviados desde Postman: la publicación con el contenido ya editado por el PATCH y el comentario con 1 like, resultado del `$inc`.

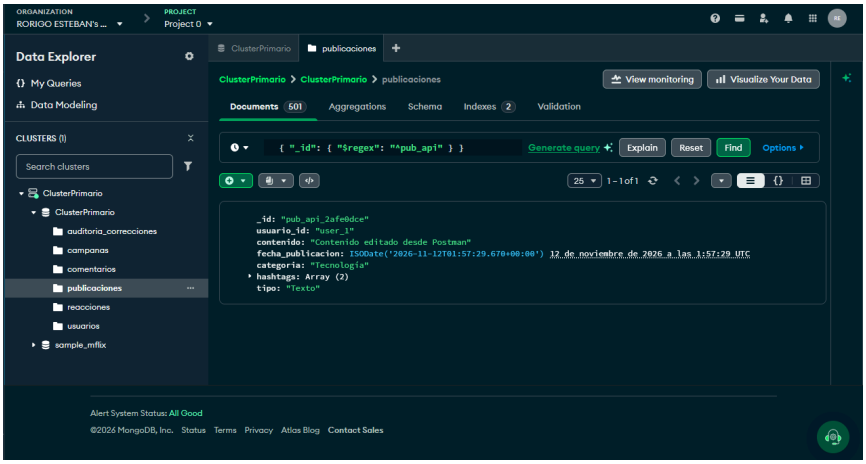


Figura 19. Atlas, colección publicaciones (501 documentos): publicación pub_api_2afe0dce creada desde Postman y editada con PATCH.

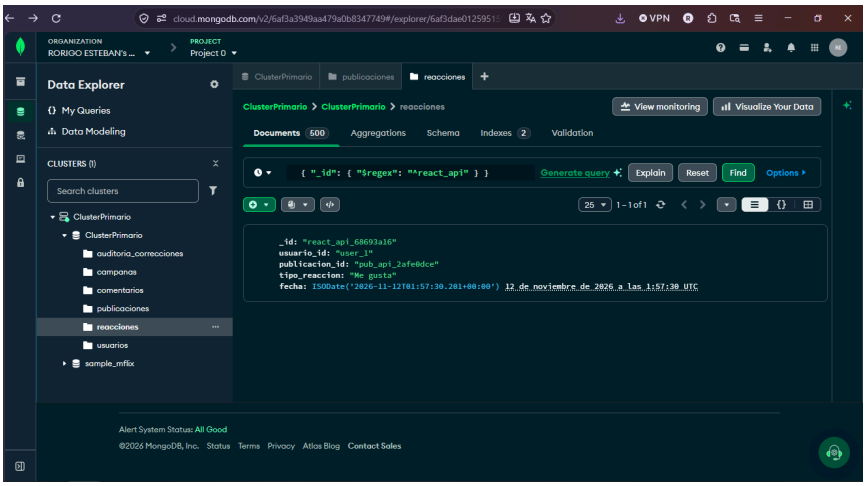


Figura 20. Atlas, colección reacciones (500 documentos): react_api_68693a16 creada desde Postman.

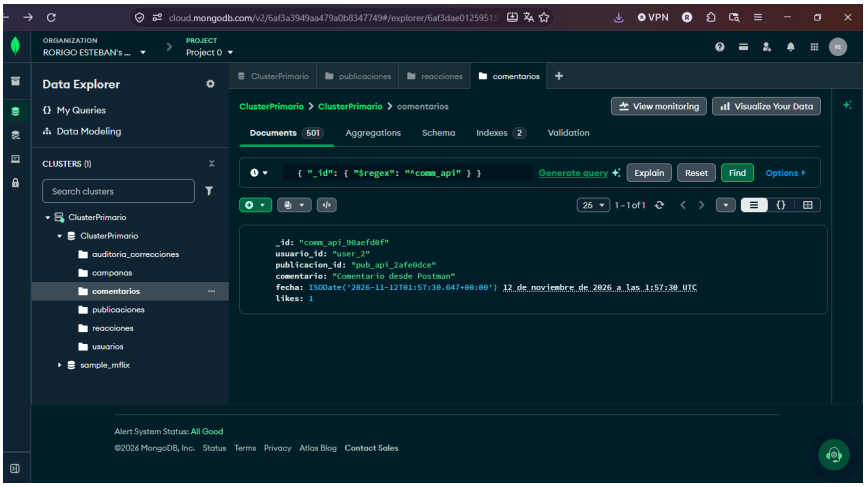


Figura 21. Atlas, colección comentarios (501 documentos): comm_api_90aefd0f con likes = 1.

Luego se ejecutaron las solicitudes de eliminación (14, 15 y 16). Los mismos filtros devuelven "No results" y los conteos vuelven al estado original, lo que confirma que la API también elimina en tiempo real y no deja datos huérfanos:

Colección	Con los datos de Postman	Después de eliminarlos	Estado original
publicaciones	501	500	500
reacciones	500	499	499
comentarios	501	500	500

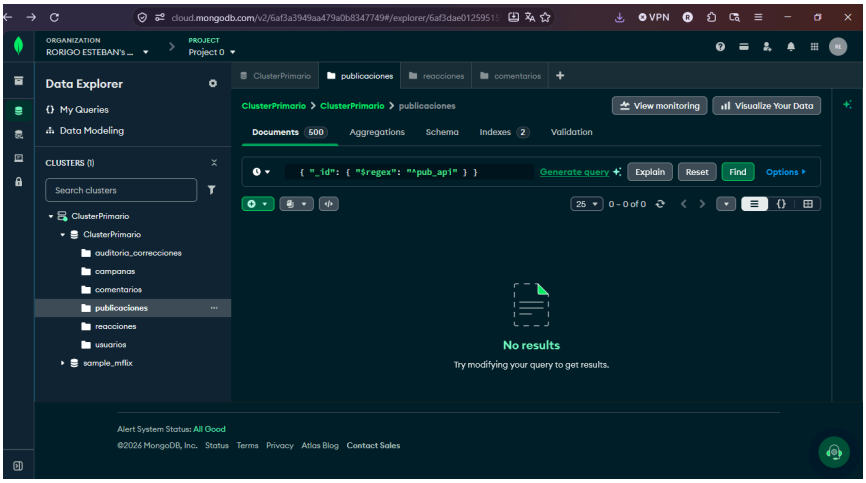


Figura 22. Atlas, publicaciones después de eliminar: el filtro pub_api no devuelve resultados y la colección vuelve a 500 documentos.

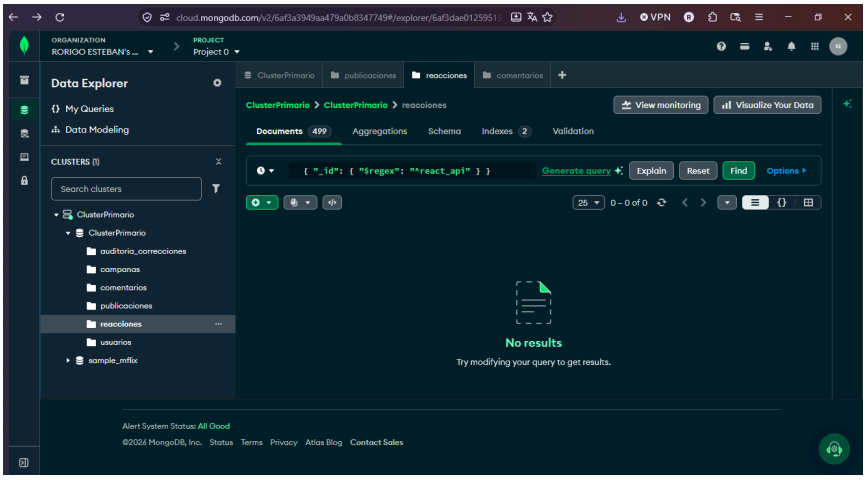


Figura 23. Atlas, reacciones después de eliminar: sin resultados y 499 documentos.

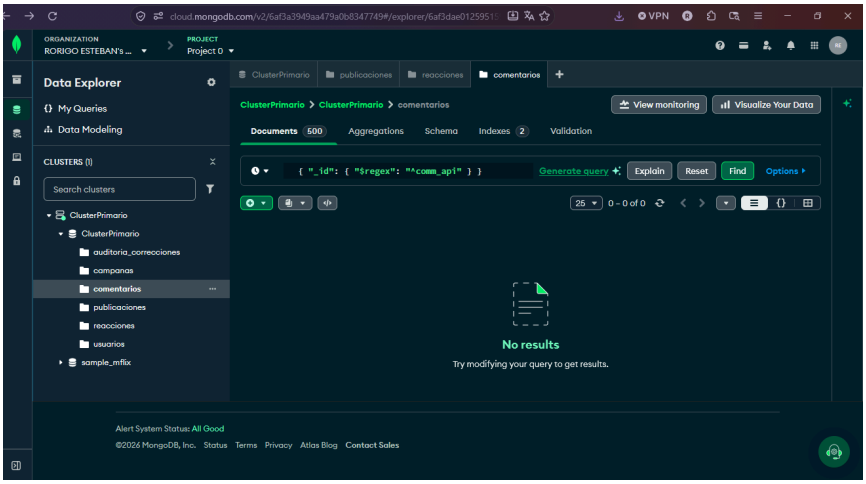


Figura 24. Atlas, comentarios después de eliminar: sin resultados y 500 documentos.

Tiempos de respuesta. Según el registro de la API, las operaciones tardan entre 55 y 250 ms, que corresponde a la ida y vuelta hacia el cluster en São Paulo. Las excepciones son la primera conexión (cerca de 1 s) y las validaciones rechazadas antes de consultar la base, que responden en menos de 1 ms.

7. Control de errores y validaciones

7.1 Validación de entradas

Regla	Dónde se aplica	Respuesta
El cuerpo debe ser un objeto JSON	Todos los POST y PATCH	400
Campos obligatorios presentes y no vacíos	Publicaciones, reacciones y comentarios	400, indicando el campo que falta
Tipos y largo máximo de los textos (hasta 500 caracteres)	Contenido y comentario	400
Valores dentro de los permitidos (categoría, tipo y tipo de reacción)	Publicaciones y reacciones	400, con la lista de valores permitidos
Hashtags como lista de textos que comienzan con #	Publicaciones	400
Parámetro `limit` entero entre 1 y 100	Listados	400
El usuario y la publicación referenciados deben existir	Publicaciones, reacciones y comentarios	422
No puede haber dos reacciones del mismo usuario sobre una publicación	Reacciones	409
El documento buscado debe existir	Lectura, edición, like y eliminación	404

7.2 Manejo de excepciones y códigos HTTP

Los errores se devuelven siempre con el mismo formato JSON (`codigo`, `mensaje` y, cuando corresponde, `detalle`). La API distingue los errores del cliente de los del servidor y de la base de datos:

Código	Situación	Mensaje observado en las pruebas
400	Datos inválidos o faltantes	Falta el campo obligatorio 'contenido'
404	Documento o ruta inexistente	La publicación 'pub_inexistente' no existe
409	Clave duplicada	El usuario ya reaccionó a esta publicación
422	Referencia inválida	Referencia inválida: la publicación 'pub_inexistente' no existe
503	Sin conexión con la base de datos o variable de entorno ausente	No se pudo conectar con la base de datos (implementado; no se provocó en las pruebas)
500	Error de MongoDB o error interno	Error al operar con la base de datos / Error interno del servidor

Los códigos 400, 404, 409 y 422 se verificaron con la colección de Postman (solicitudes 06, 07, 11 y 12) y quedaron registrados en el log de la API.

7.3 Registros de actividad (logs)

Cada solicitud queda registrada en `logs/api.log` con método, ruta, código de estado y duración. Además se registran las operaciones de negocio (creación y eliminación de documentos) y las advertencias de los errores controlados. El siguiente extracto corresponde a la ejecución cuyos resultados se muestran en la sección 6.4:

```
22:57:29,277 [INFO] GET /health -> 200 (55 ms)
22:57:29,599 [INFO] GET /publicaciones?limit=3 -> 200 (72 ms)
22:57:29,787 [INFO] Publicación creada: pub_api_2afe0dce por user_1
22:57:29,788 [INFO] POST /publicaciones -> 201 (118 ms)
22:57:30,265 [INFO] Reacción creada: react_api_68693a16
22:57:30,265 [INFO] POST /reacciones -> 201 (238 ms)
22:57:30,470 [WARNING] ApiError 409: El usuario ya reaccionó a esta publicación
22:57:30,470 [INFO] POST /reacciones -> 409 (174 ms)
22:57:30,617 [WARNING] ApiError 422: Referencia inválida: la publicación 'pub_inexistente' no existe
22:57:30,617 [INFO] POST /reacciones -> 422 (117 ms)
22:57:30,822 [INFO] Comentario creado: comm_api_90aefd0f
22:57:30,905 [INFO] POST /comentarios/comm_api_90aefd0f/like -> 200 (58 ms)
22:57:31,082 [WARNING] ApiError 404: La publicación 'pub_inexistente' no existe
22:57:31,082 [INFO] GET /publicaciones/pub_inexistente -> 404 (56 ms)
22:57:31,110 [WARNING] ApiError 400: Falta el campo obligatorio 'contenido'
22:57:31,110 [INFO] POST /publicaciones -> 400 (0 ms)
00:10:25,912 [INFO] Publicación eliminada: pub_api_2afe0dce (reacciones=0, comentarios=0)
00:10:25,912 [INFO] DELETE /publicaciones/pub_api_2afe0dce -> 200 (247 ms)
```

El registro también incluye solicitudes automáticas del navegador, como `GET /favicon.ico -> 404`, que son inofensivas.

7.4 Prueba de conectividad con la base de datos

La API expone `GET /health`, que ejecuta un `ping` a Atlas y devuelve el estado, el nombre de la base y la latencia (ver sección 2.7). Esto permite comprobar en cualquier momento que la comunicación entre la aplicación y la base funciona. El script `01_verificar_conexion.py` cumple el mismo propósito desde la línea de comandos.

Anexo. Estructura del repositorio

Repositorio: https://github.com/YamikugoKurogane/contentflow-atlas. Las credenciales se guardan solo en un archivo `.env` local, que no se sube; el repositorio incluye `.env.example` sin datos reales.

Archivo	Función	Sección
`scripts/01_verificar_conexion.py`	Prueba de conectividad y concerns	2.7
`scripts/02_importar_datos.py`	Importación de los cinco JSON	3.1
`scripts/03_convertir_fechas.py`	Conversión de fechas a Date	3.2
`scripts/04_validar_datos.py`	Detección de inconsistencias	5.1
`scripts/05_corregir_datos.py`	Corrección y marcado de datos	5.2 y 5.3
`scripts/06_crear_indices.py`	Creación de los 9 índices	2.4
`scripts/07_consultas.py`	5 consultas de lectura y 8 agregaciones	4.1 a 4.3
`scripts/08_explain_indices.py`	Medición con explain antes y después de los índices	4.4
`scripts/09_crud.py`	Caso CRUD con validaciones	5.4
`api/app.py`	API REST con Flask	6 y 7
`postman/ContentFlow_API.postman_collection.json`	Colección de Postman exportada	6.4
`docs/evidencias/*.json`	Salida en JSON de cada script	Todas
`.env.example`, `README.md`	Configuración y guía de ejecución	-